

Template de rapport de gestion de projet

Introduction

En tant qu'ingénieur data freelance spécialisé en bases de données vectorielles et en intégration de modèle NLP, l'objectif est d'accompagner l'entreprise Puls-Events dans la mise en place d'un chatbot personnalisé, sur leur site web, pour aider les utilisateurs dans la recherche d'évènements culturels divers et variés.

Le but est donc de proposer ce chatbot avec un système de génération de réponses augmentées (RAG), qui doit être capable de récupérer, analyser et proposer une solution pertinente grâce aux données récupérées depuis une base de données vectorielle.

Pour représenter de manière concrète ce projet, un POC (Proof Of Concept) a été dessiné, réalisé et présenté. Ce POC a permis de visualiser l'architecture globale de ce chatbot en incluant les notions de RAG (Retrieval-augmented Generation), base de données vectorielle, de modèle d'IA, etc.

Ce POC a été validé dans son ensemble pour permettre de se lancer vers la prochaine étape, le MVP (Minimum Viable Product).

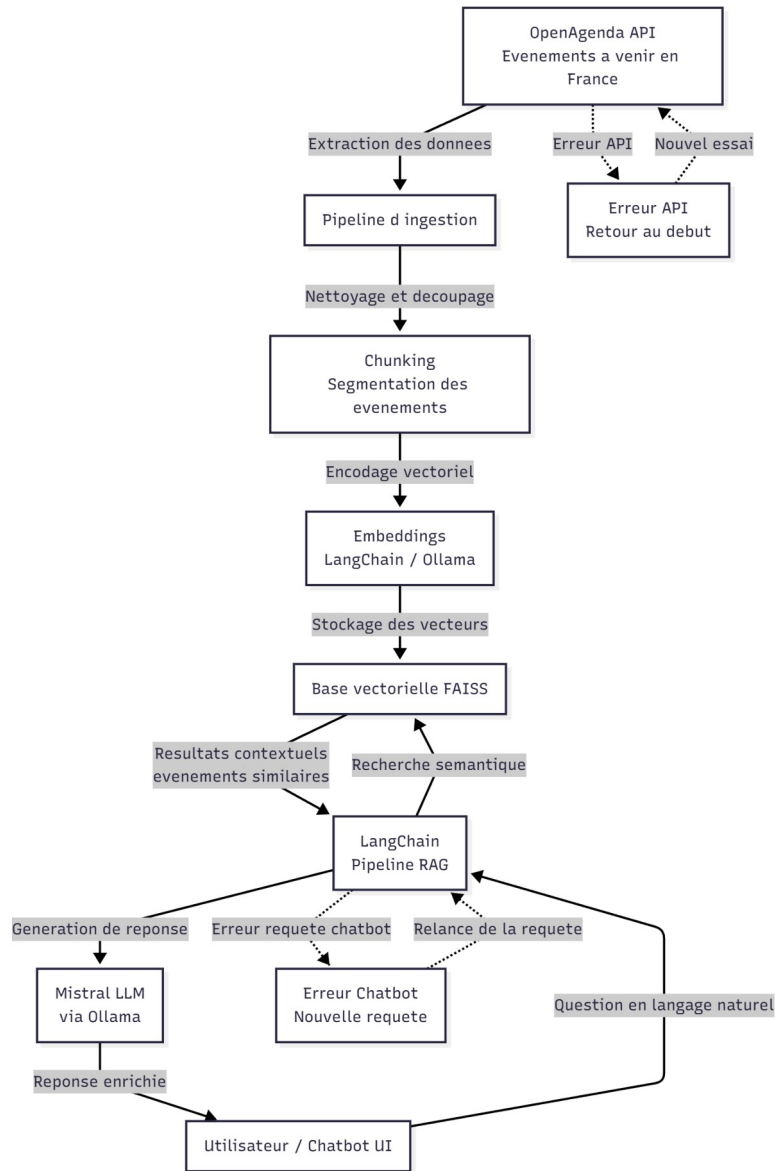
L'idée est de récupérer les besoins de l'équipe afin de partir d'une base existante et fonctionnelle (POC) et de l'améliorer avec de nouvelles fonctionnalités afin de le faire tendre vers le format le plus abouti (MVP).

Pour ce faire, il faut cadrer les améliorations à apporter à ce POC, les estimer, les planifier, afin de définir une date limite réaliste, des coûts estimés prévisionnels et une architecture claire à l'avance. Fort de ces objectifs, ce rapport de gestion de projet a été réalisé.

1. Analyse et synthèse des besoins formulés par l'équipe

1.1 Synthèse du contexte, objectifs métiers et contraintes techniques

Afin de faire une synthèse du contexte cohérente et compréhensible, des objectifs métiers et des contraintes techniques, voici, pour rappel, l'architecture mise en place lors du POC :



Ce diagramme décrit l'architecture et le fonctionnement du chatbot dans son format POC.

Pour rappel, l'objectif global est de récupérer, traiter, stocker et exploiter les données publiques d'événements culturels fournis par l'API publique d'Open Agenda, afin de pouvoir fournir des informations en langage naturel aux utilisateurs.

Première étape (Récupération des données depuis OpenAgenda) :

La première étape consiste à récupérer les données depuis l'API publique d'OpenAgenda. Pour réaliser cette étape, il fallait étudier la documentation API fournie par OpenAgenda et écrire un script Python permettant de récupérer les informations nécessaires, en prenant en compte les limites de récupération de données indiquées par la documentation.

En prévision de potentiels problèmes liés aux appels API, une mécanique consistant à relancer les appels est mise en place au cas où des erreurs surviennent.

De plus, c'est directement depuis l'appel API que les paramètres géographiques et de temps sont passés.

Seconde étape (Pipeline d'ingestion et préparation des données) :

La seconde étape consiste à traiter les données récupérées depuis OpenAgenda avant de réaliser l'encodage vectoriel et de les stocker.

L'idée de ce nettoyage est de supprimer les potentiels doublons qui pourraient s'être glissés à l'intérieur des données récupérées. Pour repérer les doublons, il suffit de comparer, le nom de l'évènement, sa période et son lieu.

C'est aussi ici que l'on vient segmenter les évènements (chunking) en prévision de leur intégration dans la base de données vectorielle.

Troisième étape (Encodage vectoriel et stockage) :

La troisième étape consiste à réaliser un encodage vectoriel des données et de les stocker. Ce qu'il se passe est que chaque chunk récupéré de l'étape précédente va être converti en vecteur numérique grâce à un modèle d'embeddings via Ollama. Ollama est un outil open-source permettant de récupérer des modèles d'IA en local (on-premise) et de ne pas à avoir à utiliser l'IA via API.

Une fois les vecteurs numériques récupérés, ils sont stockés dans ce qu'on appelle une base de données vectorielle, ici, il s'agit d'une base FAISS (Facebook AI Similarity Search), qui est une base optimisée pour la recherche de similarité.

Quatrième étape (Recherche sémantique et RAG) :

Lorsqu'un utilisateur pose une question au chatbot. LangChain récupère cette requête et interroge la base de données vectorielle FAISS, qui effectue une recherche sémantique pour retrouver les informations les plus pertinentes.

Une fois ces informations récupérées, il vient les envoyer au LLM.

Cinquième étape (Génération de la réponse) :

Une fois le contexte récupéré depuis LangChain, le LLM va venir réorganiser ces informations et les fournir en langage naturel à l'utilisateur afin d'avoir l'impression de parler avec un être humain. L'idée est de faire en sorte que ce soit compréhensible pour lui et surtout, pertinent.

L'idée est qu'il combine, question d'origine, réponse à la question et uses et coutumes de discussion pour donner la réponse la plus claire et naturelle.

Aussi, afin d'observer des résultats cohérents, il faut faire attention à la manière dont est configuré le prompt pour qu'il donne les meilleurs résultats possibles. De

plus, il faut également jouer avec la température du model afin de le rendre plus ou moins stricte vis à vis des réponses qu'il fournit.

1.2 Analyse des utilisateurs cibles et des cas d'usage

Il est possible de distinguer, à première vue, 2 types d'utilisateurs cibles avec des besoins et des attentes différentes, mais surtout des profils différents au sein de ces types d'utilisateurs.

Le premier est l'**utilisateur occasionnel** :

Contexte :

L'utilisateur occasionnel consulte des événements de manière ponctuelle. On y retrouve les touristes, ou les habitants qui sortent de manière occasionnelle, pendant des week-end ou pour des événements particuliers.

Objectifs :

Son objectif est de trouver rapidement des événements qui puissent lui correspondre, sans qu'il passe des heures à faire des recherches et sans forcément connaître les usages et coutumes de son environnement.

Besoins :

Ce type d'utilisateur vient chercher un outil simple d'utilisation, avec des propositions attrayantes au travers de réponses claires, synthétiques et dynamiques.

Cas d'usages :

« Donne moi des événements culturels à venir sur Toulouse »
« Que faire sur Paris ce soir ? »

Le second est l'**utilisateur régulier** :

Contexte :

L'utilisateur régulier peut être un voyageur chevronné, ou un habitant qui sort, fait des activités, de manière régulières.

Objectifs :

L'objectif de cet utilisateur va être de trouver des événements qui correspondent à des domaines bien précis, ou des lieux peu fréquentés, connus d'un nombre d'utilisateurs restreint.

Besoins :

Ce type d'utilisateur aura besoin d'une aide fine et approfondie sur ses demandes. Il faudra que le chatbot ait la connaissance d'un champ lexical artistique précis afin de répondre, au plus précis à ce type d'utilisateur.

Cas d'usage :

« Il y a-t-il un événement musical de Spytrack sur Toulouse à venir ? »

« Quelle est la prochaine exposition d'impressionnisme à venir autour de chez moi ? »

1. Macro backlog des fonctionnalités

Un macro backlog est une liste de fonctionnalités structurées et priorisées.

L'objectif est d'estimer le temps de réalisation de ces tâches afin d'avoir une idée concrète du temps de réalisation globale du projet.

L'idée est de découper les grandes parties des évolutions demandées en tâches.

Toutes les tâches se voient attribuées une estimation que l'on va appeler (estimation basse).

Pour chaque tâche, un indice de complexité va être défini entre 0 et 5. Cet indice de complexité va être utilisé pour servir de référence de majoration. Pour chaque indice de complexité, une nouvelle estimation sera calculée selon cette formule :

$\text{Estimation basse} \times (1 + \text{Taux de majoration lié à la complexité})$
--

De plus, chaque sous tâche est définie sur « Must have » ou « Nice to have » en fonction de l'importance de la tâche dans le cadre du MVP.

Must have = Fonctionnalité requise pour respecter le cahier des charges dans le cadre du MVP.

Nice to have = Fonctionnalité intéressante mais non obligatoire pour le bon fonctionnement du chatbot dans le cadre du MVP.

2. Plan de projet

Le macro planning du projet a été construit à partir des estimations issues du macro backlog fonctionnel. Pour chaque grande partie du projet, les charges estimées (en jours) ont été consolidées afin d'obtenir une vision globale de l'effort nécessaire à la réalisation du MVP.

Ces estimations ont ensuite été projetées sur un découpage temporel en semaines, permettant de positionner les grandes phases du projet dans le temps. L'objectif de ce macro planning n'est pas de détailler chaque tâche à un niveau opérationnel, mais de fournir une vision synthétique et lisible de l'enchaînement des travaux.

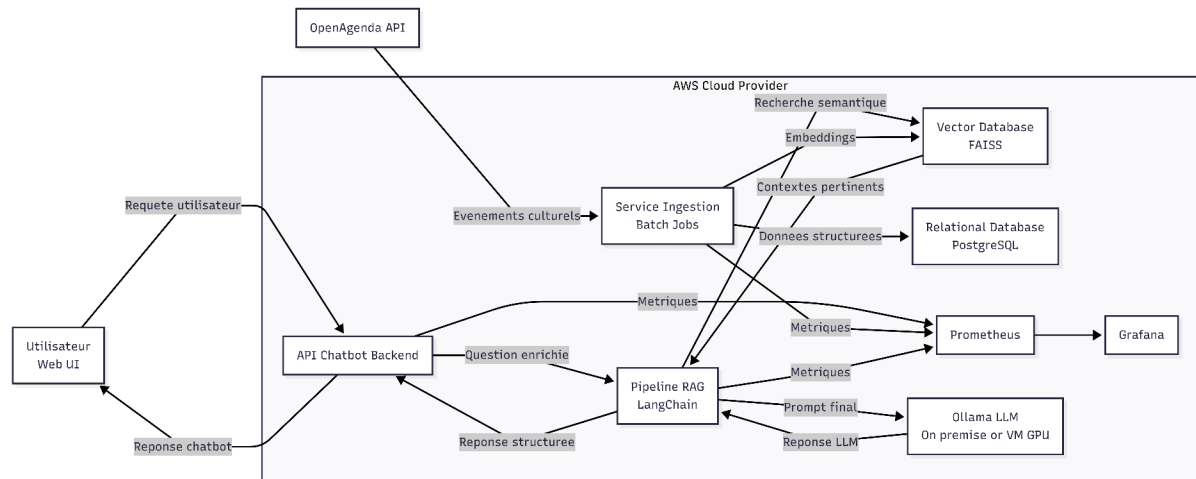
Le planning met ainsi en évidence :

- La répartition des différentes parties du projet dans le temps,
- Les chevauchements possibles entre certaines activités,
- Les périodes clés correspondant aux jalons et livrables majeurs.

Cette approche permet de disposer d'un outil de pilotage simple, facilitant la communication autour du projet et l'anticipation des charges, tout en laissant une marge de flexibilité pour les ajustements nécessaires au cours de la réalisation.

3. Architecture technique détaillée

3.1 Schéma d'architecture cloud scalable



3.2 Choix du cloud provider et justification de la décision par une veille technique

Démarche :

Afin de choisir le bon cloud provider, il faut réaliser sa veille et ses recherches techniques sur des points bien précis :

- Coût de la solution
- Scalabilité de la solution
- Facilité d'utilisation et de configuration de la solution
- Proposition de solutions modernes (ex : IA)

Le but est de visualiser le meilleur outil selon ces critères et un comparatif entre différentes solutions du marché.

Résultat de veille :

Plusieurs critères ont été retenus auprès de la solution AWS car ils collaient avec les critères de recherches initiaux de la veille (Amazon Web Services) :

- Pertinence dans l'utilisation de l'IA au sein de l'écosystème
- Scalable et haute disponibilité
- Déploiement facilité avec les notions de conteneurs, VM, etc.
- Gestion des coûts (OPEX (Operational Expenditure)) et granularité de la facturation
- Intégration avec des solutions open-source
- Communauté active donnant lieu à de la documentation complémentaire avec des cas concrets

Analyse comparative des cloud provider :

AWS :

- Leader du cloud avec une offre très large
- Forte compatibilité avec des architectures modulaires (comme dans le cadre du Chatbot)
- Large choix de services (sous forme de modules) pour le calcul, le stockage et le monitoring
- Documentation riche, communauté active et donc une large capacité de support

Microsoft Azure :

- Très bonne intégration au sein d'un environnement microsoft (pas pertinent dans notre cas)
- Services IA aussi très performants et novateurs
- Malheureusement moins flexible pour des architectures open source et on-premise comme le chatbot avec la volonté d'avoir le reste de l'architecture on-premise (ex : Ollama)

Google Cloud Platform :

- Excellente expertise sur les sujets data et ML (machine learning) notamment
- Services très performants mais pas très orientés expérience utilisateur (interface et solution plus complexe à prendre en main)

Conclusion sur le choix du cloud provider :

Pour conclure, le choix s'est donc porté sur AWS pour les raisons suivantes :

- **Fiabilité de la solution** : AWS dispose d'un écosystème éprouvé, capable de supporter une montée en charge progressive du chatbot.
- **Architecture flexible** : AWS permet de déployer aussi bien des services conteneurisés que des machines virtuelles, facilitant l'intégration d'un moteur LLM hybride (on-prem ou cloud).
- **Open-source compatible** : Les briques techniques du projet (LangChain, FAISS, Ollama, Prometheus, Grafana) s'intègrent naturellement dans l'environnement AWS.
- **Gestion des coûts** : la granularité de la facturation permet de démarrer avec des ressources limitées et d'ajuster progressivement en fonction de l'usage réel.

3.3 Stratégie de déploiement, de modularité et de monitoring

Déploiement :

L'idée est de faire un déploiement progressif et industrialisable. Les composants du chatbot seraient déployés indépendamment les uns des autres, exécutés dans un environnement Cloud AWS.

Le but de cette approche est de proposer une mise en production rapide du chatbot, une montée en charge progressive et isoler les responsabilités entre les différents services.

Pour automatiser les déploiements, il faudra passer par des pipelines CI/CD qui vont valoriser la reproductibilité des environnements, la réduction des erreurs humaines en limitant les actions manuelles et enfin, la possibilité de déployer des mises à jour sans interruption de service.

Modularité :

Le chatbot est conçu nativement de manière modulaire. Voici les principaux modules identifiés :

- Le backend (API)
- Le pipeline RAG
- Le service d'ingestion des données
- La base de donnée vectorielle
- Le moteur LLM

Cette modularité permet :

- De faire évoluer chaque composant indépendamment
- De remplacer une brique technologique sans refonte globale (ex : changement de modèle LLM)
- De faciliter les tests, la maintenance et l'évolution fonctionnelle.

L'idée est de limiter l'impact d'une défaillance sur le fonctionnement global du chatbot en isolant les différentes briques.

Monitoring :

La stratégie de Monitoring va permettre de :

- Assurer la performance technique du système
- Assurer la qualité de service perçue par les utilisateurs
- Assurer l'amélioration continue du chatbot

Prometheus va permettre de collecter des statistiques au travers de métriques, de manière centralisée, qui couvriront :

- Les temps de réponse
- Les taux d'erreurs
- L'utilisation des ressources
- Les volumes de requêtes

Enfin Grafana sera utilisé pour mettre en forme sous forme de graphiques et de dashbaord, les métriques paramétrées par Prometheus :

- Supervision en temps réel
- L'analyse des usages
- L'identification rapide des anomalies

4. Estimation des coûts build & OPEX :

Coûts build :

Les coûts de build correspondent à l'ensemble des coûts de production liés à la phase de conception et de réalisation du projet. Cette phase couvre le développement des fonctionnalités nécessaires à la mise à disposition d'un chatbot fonctionnel au format MVP.

Dans le cadre de ce projet, les coûts de build sont directement liés au temps de développement des différentes fonctionnalités identifiées dans le macro backlog, incluant notamment la mémoire conversationnelle, le contexte géographique, la recherche web et le monitoring.

L'estimation repose sur les hypothèses suivantes :

- Les charges sont évaluées en jours de développement,
- Les estimations hautes sont privilégiées afin d'intégrer les incertitudes liées à la complexité technique,
- Une base de calcul unique est utilisée, le TJM (Taux Journalier Moyen).

Le TJM représente le coût journalier d'une ressource technique (développeur, ingénieur data ou architecte) mobilisée sur le projet. Cette approche permet :

- D'obtenir une estimation homogène et lisible,
- De faciliter les ajustements budgétaires en cas de modification du périmètre,
- De garantir une vision réaliste des efforts nécessaires à la construction du MVP.

OPEX :

Les OPEX (coûts d'exploitation) correspondent aux dépenses nécessaires au fonctionnement continu du chatbot une fois la phase de build terminée. Ils couvrent l'ensemble des ressources techniques, humaines et infrastructurelles mobilisées pour garantir la disponibilité, la performance et la qualité du service. Dans une logique MVP, l'objectif n'est pas de surdimensionner l'infrastructure, mais de dimensionner au plus juste en fonction des charges utilisateurs prévues.

Les coûts OPEX prennent ainsi en compte :

- L'hébergement de l'infrastructure cloud (compute, stockage, réseau),
- L'exploitation des services de monitoring,
- Les activités de supervision et de maintenance applicative.

Le choix d'une architecture cloud modulaire et scalable permet :

- D'ajuster les ressources en fonction de l'usage réel,
- De limiter les coûts fixes,
- De transformer une partie des coûts en charges variables.

Optimisations :

Dès la phase de conception, une attention particulière a été portée à l'identification de leviers d'optimisation budgétaire, afin de limiter les coûts sans compromettre la qualité du service.

Plusieurs axes d'optimisation ont été retenus :

- Le recours à des technologies open source pour limiter les coûts de licences,

- Une architecture hybride pour l'hébergement du moteur LLM, permettant de différer l'usage de ressources GPU cloud coûteuses,
- Une ingestion des données réalisée de manière batch afin de réduire la consommation de ressources en continu,
- Une stratégie de monitoring ciblée, évitant la collecte excessive de métriques et de logs.

Ces choix permettent de réduire significativement les coûts d'exploitation du MVP, tout en conservant une architecture évolutive capable de s'adapter à une augmentation du nombre d'utilisateurs ou à de nouveaux besoins fonctionnels.

5. BILAN :

5.1 Rappel des étapes clés du POC au MVP

Le projet a débuté par une phase de POC (Proof of Concept) visant à valider la faisabilité technique du chatbot et la pertinence de l'utilisation des données issues d'OpenAgenda pour la recommandation d'événements culturels.

Cette première étape a permis de :

- Confirmer la capacité du chatbot à interroger une API externe,
- Tester un premier pipeline de traitement et de restitution des données,
- Valider l'intérêt fonctionnel auprès des équipes.

À l'issue du POC, une phase d'analyse des besoins a été menée afin d'identifier les évolutions nécessaires au passage vers un MVP (Minimum Viable Product). Cette phase s'est appuyée sur des retours utilisateurs internes et sur l'anticipation des futurs usages.

Le passage du POC au MVP s'est traduit par :

- La formalisation d'un macro backlog fonctionnel,
- L'introduction de mécanismes de personnalisation (mémoire conversationnelle, contexte géographique, recherche web),
- La prise en compte des enjeux d'exploitation (monitoring, performance, coûts),
- La structuration du projet autour d'une architecture cloud scalable.

5.2 Justification des choix techniques et méthodologiques

Les choix techniques et méthodologiques réalisés tout au long du projet ont été guidés par un objectif central : construire un MVP robuste, évolutif et économiquement maîtrisé.

Sur le plan technique, l'adoption d'une architecture cloud modulaire permet :

- De découpler les différents composants du système,

- De faciliter les évolutions futures,
- De garantir une montée en charge progressive.

Le recours à des technologies open source (LangChain, FAISS, Prometheus, Grafana) répond à une double logique :

- Limiter les coûts de licences,
- Conserver une indépendance vis-à-vis des fournisseurs.

Sur le plan méthodologique, la structuration du projet autour :

- D'un macro backlog,
- D'estimations majorées par un indice de complexité,
- D'un macro planning,

a permis de sécuriser la planification tout en conservant une marge de manœuvre pour les ajustements nécessaires.

5.3 Défis rencontrés et solutions apportées

On peut diviser en 3 parties les défis rencontrés :

- Dans un premier temps, d'un point de vue technique. Comment faire pour estimer les temps sur les tâches en sachant qu'il n'y a que 4 grandes tâches et donc un contexte assez vague.

L'idée pour concurrencer ce problème, a été de diviser chaque grande tâche en sous-tâche afin de faciliter l'appréciation de celles-ci et d'avoir une facilité d'estimation.

- Dans un second temps, d'un point de vue coût. L'idée étant de mettre en place une estimation des coûts sur un potentiel outil cloud, car pas encore décidé. De plus, le poste n'étant pas encore créé, il est difficile de donner des estimations de coûts précises.

L'idée est donc de fixer un TJM estimatif pour essayer de gérer les coûts sur sa base et de définir un cloud provider pour définir des coûts basés sur celui-ci.

- Dans un troisième temps, comment avoir du recul sur le chatbot une fois passé en MVP. Pour validation du POC, il y a eu une première présentation du projet, maintenant, comment faire pour avoir des retours sur le chatbot une fois déployé en production.

C'est au vu de ces problématiques que les outils de monitoring ont été définis et que la récupération de compléments d'informations des utilisateurs a été préconisé afin d'avoir plus de visibilité sur les axes d'amélioration du chatbot et de son utilisation.

6. CONCLUSION:

Ce rapport a permis de présenter la transformation progressive du chatbot d'événements culturels, initialement conçu sous forme de POC, vers un MVP structuré et exploitable. À travers l'analyse des besoins, la définition d'un macro backlog, l'élaboration d'un macro planning et l'estimation des coûts build et OPEX, le projet a été abordé sous un angle à la fois technique, méthodologique et économique. Les choix architecturaux réalisés, notamment autour d'une infrastructure cloud scalable et modulaire, visent à garantir la pérennité et l'évolutivité de la solution.

L'intégration des enjeux de monitoring, de performance et de maîtrise budgétaire dès la phase de conception témoigne d'une approche orientée produit.

Enfin, ce travail met en évidence l'importance d'un pilotage rigoureux pour sécuriser le passage du prototype à un service à valeur ajoutée, capable de s'adapter aux usages réels et aux évolutions futures.

7. PORTFOLIO :

Suite aux demandes liées à ce projet. Une ouverture de poste est prévue au sein de Puls-Events.

Souhaitant proposer ma candidature, vous pourrez retrouver mon portfolio en pièces jointes de ce document au format PDF (et sur le repo github du projet) récapitulant :

- Projets réalisés en tant que Data Engineer
- Technologies expérimentées
- Compétences acquises

Merci pour cette opportunité.